



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра информационных технологий в атомной энергетике

Практическая работа 2

по дисциплине «Информационные системы управления данными и документами»

Студент группы ИКБО-50-23

Враженко Д. О.

(подпись студента)

Принял преподаватель

Тесла-Блозва С. А.

(подпись руководителя)

Работа представлена

«___» _____ 2026 г.

Допущен к работе

«___» _____ 2026 г.

Москва 2026

Оглавление

1. Цель работы.....	3
2. Ход выполнения работы.....	3
2.1 Часть 1. Создание проекта на Start.Vaadin.....	3
2.2 Часть 2. Создание класса модели Document.....	4
2.3 Часть 3. Создание репозитория и сервиса.....	5
2.4 Создание экранной формы (View).....	7
2.5 Часть 5. Тестирование приложения.....	9
3. Вывод.....	10
4. Ответы на контрольные вопросы.....	10

1. Цель работы

Спроектировать экранную форму создания документа, настроить связывание данных с моделью и реализовать логику проверки вводимых значений.

2. Ход выполнения работы

2.1 Часть 1. Создание проекта на Start.Vaadin

Генерация проекта на сайте <https://start.vaadin.com/app?type=sample> представлена на Рисунке 1.

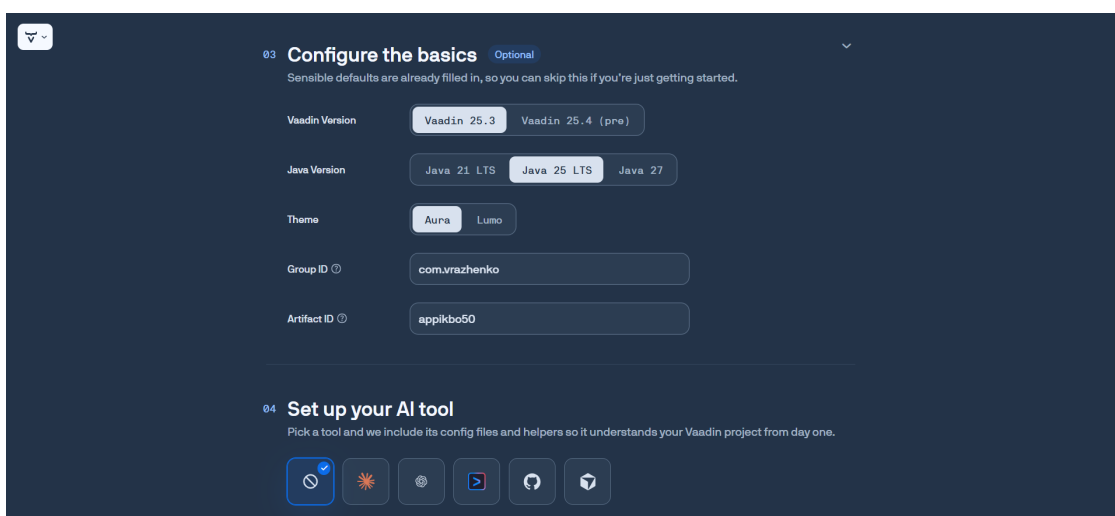


Рисунок 1 – Генерация проекта на сайте vaadin

Результат успешного запуска проекта представлен на Рисунке 2.

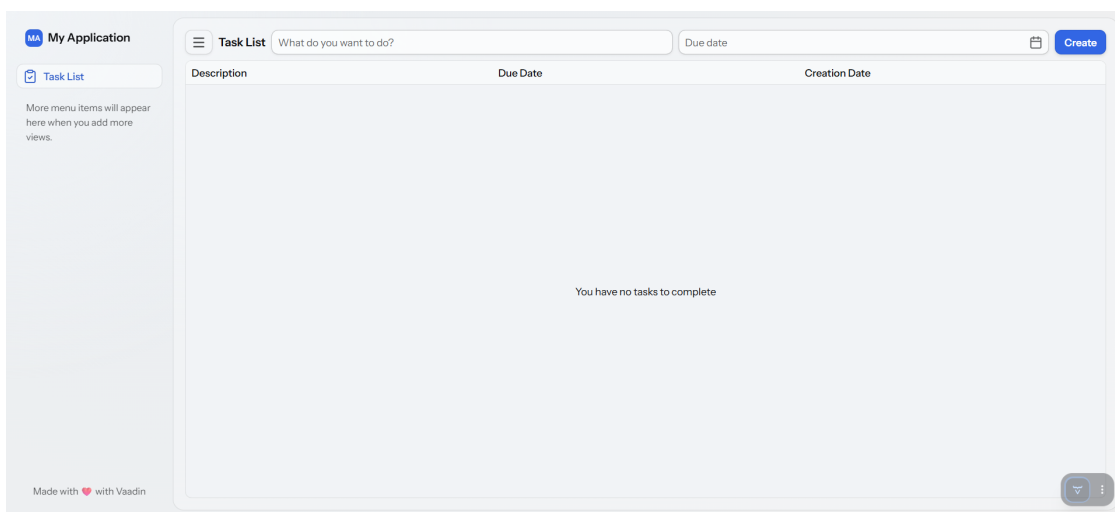


Рисунок 2 – Результат проверки доступности порта 5432

2.2 Часть 2. Создание класса модели Document

Содержание класса Document.java представлено в листинге ниже:

```
package com.vrazhenko.documentfeature;

import java.time.LocalDate;

public class Document {
    private String title;
    private String regNumber;
    private LocalDate regDate;
    private String type;
    private boolean urgent;

    public Document() {
    }

    public Document(String title, String regNumber, LocalDate regDate, String
type, boolean urgent) {
        this.title = title;
        this.regNumber = regNumber;
        this.regDate = regDate;
        this.type = type;
        this.urgent = urgent;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getRegNumber() {
        return regNumber;
    }

    public void setRegNumber(String regNumber) {
        this.regNumber = regNumber;
    }
}
```

```

    public LocalDate getRegDate() {
        return regDate;
    }

    public void setRegDate(LocalDate regDate) {
        this.regDate = regDate;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public boolean isUrgent() {
        return urgent;
    }

    public void setUrgent(boolean urgent) {
        this.urgent = urgent;
    }

    @Override
    public String toString() {
        return "Document{" +
            "title='" + title + '\'' +
            ", regNumber='" + regNumber + '\'' +
            ", regDate=" + regDate +
            ", type='" + type + '\'' +
            ", urgent=" + urgent +
            '}';
    }
}

```

2.3 Часть 3. Создание репозитория и сервиса

Содержание интерфейса `DocumentRepository.java` представлено в ЛИСТИНГЕ НИЖЕ:

```
package com.vrazhenko.documentfeature;
```

```

import org.springframework.stereotype.Repository;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class DocumentRepository {
    private final List<Document> documents = new ArrayList<>();

    public Document save(Document document) {
        documents.add(document);
        return document;
    }

    public List<Document> findAll() {
        return documents;
    }

    public Optional<Document> findById(Long id) {
        return Optional.empty();
    }
}

```

Содержание интерфейса DocumentService.java представлено в листинге ниже:

```

package com.vrazhenko.documentfeature;

import org.springframework.stereotype.Service;

@Service
public class DocumentService {
    private final DocumentRepository documentRepository;

    public DocumentService(DocumentRepository documentRepository) {
        this.documentRepository = documentRepository;
    }

    public Document saveDocument(Document document) {
        return documentRepository.save(document);
    }
}

```

2.4 Создание экранной формы (View)

Содержание класса `DocumentFormView.java` представлено в листинге ниже:

```
package com.vrazhenko.documentfeature.ui;

import com.vaadin.flow.component.button.Button;
import com.vaadin.flow.component.checkbox.Checkbox;
import com.vaadin.flow.component.combobox.ComboBox;
import com.vaadin.flow.component.datepicker.DatePicker;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.notification.Notification;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.data.binder.Binder;
import com.vaadin.flow.data.validator.RegexpValidator;
import com.vaadin.flow.router.Route;
import com.vrazhenko.base.ui.MainLayout;
import com.vrazhenko.documentfeature.Document;
import com.vrazhenko.documentfeature.DocumentService;
import java.time.LocalDate;

@Route(value = "document-form", layout = MainLayout.class)
public class DocumentFormView extends VerticalLayout {

    private final DocumentService documentService;
    private final Binder<Document> binder = new Binder<>(Document.class);

    private final TextField titleField = new TextField("Название");
    private final TextField regNumberField = new TextField("Регистрационный номер");
    private final DatePicker regDatePicker = new DatePicker("Дата регистрации");
    private final ComboBox<String> typeComboBox = new ComboBox<>("Тип документа");
    private final Checkbox urgentCheckbox = new Checkbox("Срочный документ");
    private final Button saveButton = new Button("Сохранить");
    private final Button clearButton = new Button("Очистить");

    public DocumentFormView(DocumentService documentService) {
```

```

        this.documentService = documentService;

        typeComboBox.setItems("Приказ", "Распоряжение", "Служебная
записка");
        typeComboBox.setPlaceholder("Выберите тип");

        FormLayout formLayout = new FormLayout();
        formLayout.add(titleField, regNumberField, regDatePicker,
typeComboBox, urgentCheckbox);
        formLayout.setResponsiveSteps(
            new FormLayout.ResponsiveStep("0", 1),
            new FormLayout.ResponsiveStep("500px", 2)
        );

        add(formLayout, saveButton, clearButton);

        binder.forField(titleField)
            .asRequired("Название не может быть пустым")
            .bind(Document::getTitle, Document::setTitle);

        binder.forField(regNumberField)
            .asRequired("Регистрационный номер обязателен")
            .withValidator(new RegexpValidator(
                "Номер должен соответствовать формату XXX-XX
(например, 123-45)",
                "^\\d{3}-\\d{2}$"))
            .bind(Document::getRegNumber, Document::setRegNumber);

        binder.forField(regDatePicker)
            .withValidator(date -> date != null && !
date.isAfter(LocalDate.now()),
                "Дата регистрации не может быть позже текущей")
            .bind(Document::getRegDate, Document::setRegDate);

        binder.forField(typeComboBox)
            .asRequired("Выберите тип документа")
            .bind(Document::getType, Document::setType);

        binder.forField(urgentCheckbox)
            .bind(Document::isUrgent, Document::setUrgent);

        saveButton.addClickListener(event -> saveDocument());

```

```

        clearButton.addClickListener(event -> clearForm());
    }

    private void saveDocument() {
        Document document = new Document();

        if (binder.writeBeanIfValid(document)) {
            System.out.println("Сохранён документ: " + document);

            documentService.saveDocument(document);

            Notification.show("Документ успешно сохранён: " +
document.getTitle());
            clearForm();
        } else {
            Notification.show("Пожалуйста, исправьте ошибки в форме",
3000, Notification.Position.MIDDLE);
        }
    }

    private void clearForm() {
        binder.readBean(new Document());
    }
}

```

2.5 Часть 5. Тестирование приложения

Проверка работы валидации при вводе некорректных данных представлена на Рисунке 3.

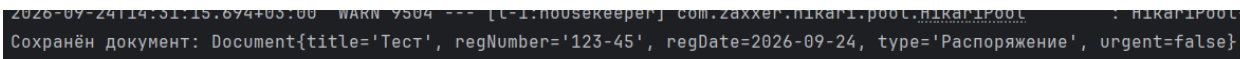
The screenshot shows a web application interface with a sidebar on the left containing 'My Application' and 'Task List'. The main form area contains several input fields with red error messages:

- Название ***: (empty). Error: ⚠ Название не может быть пустым
- Регистрационный номер ***: (12-34). Error: ⚠ Номер должен соответствовать формату XXX-XX (например, 123-45)
- Дата регистрации**: (22.05.2077). Error: ⚠ Дата регистрации не может быть позже текущей
- Тип документа ***: (dropdown menu). Error: ⚠ Выберите тип документа

Below the fields, there is a checkbox for **Срочный документ** and two buttons: **Сохранить** (disabled) and **Очистить**. At the bottom center, a notification bubble says: **Пожалуйста, исправьте ошибки в форме**. The footer of the application says 'Made with ❤ with Vaadin'.

Рисунок 3 – Результат проверки валидации

Проверка работы валидации при вводе корректных данных представлена на Рисунке 4.



```
2026-09-24T14:31:15.674+03:00 WARN 9504 --- [c-1:housekeeper] com.zaxxer.hikari.pool.HikariPool : HikariPool-  
Сохранён документ: Document{title='Тест', regNumber='123-45', regDate=2026-09-24, type='Распоряжение', urgent=false}
```

Рисунок 4 – Результат проверки валидации

3. Вывод

В результате работы было подтверждено, что серверная архитектура Vaadin упрощает создание корпоративных форм за счёт хранения состояния на сервере и автоматической синхронизации интерфейса. Полученные навыки могут быть использованы при разработке учётных систем, электронного документооборота и других бизнес-приложений, где важны валидация данных и удобство сопровождения кода на Java.

4. Ответы на контрольные вопросы

1. Что такое Vaadin и в чем главное отличие его Server-Side архитектуры от классических SPA-фреймворков (Angular, React)?

Vaadin – Java-фреймворк для веб-приложений. В Server-Side Vaadin UI-дерево и состояние компонентов хранятся на сервере, а браузер получает готовый DOM и отправляет события. В React/Angular логика и состояние находятся на клиенте, а с сервером общаются через REST/GraphQL. Vaadin позволяет писать full-stack на Java почти без JS.

2. Где физически хранится состояние дерева UI-компонентов пользовательской сессии в Vaadin?

На сервере, в VaadinSession, обычно в памяти JVM. В кластере может использоваться репликация или внешнее хранилище сессий.

3. Каким образом Spring Boot интегрируется с Vaadin и какую роль выполняет аннотация `@Route`?

Vaadin интегрируется через Spring Add-on: UI-компоненты могут быть Spring-бинами, можно инжектировать сервисы. `@Route` связывает View с URL и указывает layout/навигацию.

4. Для чего предназначен класс `Binder<T>` в экосистеме Vaadin?

Для привязки полей формы к свойствам бина: чтение, запись, валидация, отображение ошибок.

5. В чем разница между методами `binder.readBean()` и `binder.writeBean()`?

`readBean(bean)` загружает значения из бина в поля.
`writeBean(bean)` записывает значения из полей в бин и валидирует;
`writeBeanIfValid()` записывает только если валидация прошла.

6. Почему в корпоративных приложениях для создания карточек и форм рекомендуется использовать `FormLayout` вместо обычного `HorizontalLayout`?

`FormLayout` адаптивен, сам размещает поля в колонки, удобен для карточек и форм. `HorizontalLayout` просто ставит компоненты в ряд и плохо подходит для сложных форм.

7. Каким образом можно настроить кастомную валидацию пользовательского ввода (например, проверку даты или формата строки)?

Через `binder.forField(...).withValidator(...), asRequired(), RegexValidator, withConverter()`. Например, для даты:

```
date -> date != null && !
date.isAfter(LocalDate.now()).
```

8. Как реализуется динамическое изменение доступности полей интерфейса (например, перевод полей в режим «только чтение» на основе прав доступа ABAC/ACL)?

`field.setEnabled(false), setReadOnly(true), setVisible(false).` Права можно проверять через Spring Security/ACL/ABAC и применять в `beforeEnter()` или при инициализации View.

9. Для чего применяются аннотация `@Push` и метод `ui.access()` при выполнении фоновых задач?

`@Push` включает websocket-канал для обновления UI с сервера. `ui.access()` нужен для безопасного изменения UI из фонового потока, так как доступ к UI должен быть синхронизирован.

10. Назовите основные антипаттерны (распространенные ошибки разработчиков) при создании приложений на Vaadin.

- хранение больших данных в сессии/UI;
- обращение к UI из фоновых потоков без `ui.access()`;
- тяжёлая логика в конструкторе View;
- забытые listeners;
- создание компонентов через `new` вместо Spring DI;
- огромные View без декомпозиции;
- бизнес-логика во View;
- отсутствие валидации;
- проблемы с масштабированием сессий.